18

分页：简介

有一种说法是，操作系统解决几乎任何空间管理问题时，都会采取两种方法之一。第一种是把空间切成大小可变的块，就像我们在虚拟内存（virtual memory）的**分段**（segmentation）中看到的那样。遗憾的是，这种方法本身就存在一些难题。具体来说，把空间划分成大小不同的块之后，空间本身会变得**碎片化**（fragmented），分配也会随着时间推移越来越困难。

所以，也许值得考虑第二种方法：把空间切成大小固定的块。在虚拟内存中，这种思想叫作**分页**（paging），它可以追溯到一个早期的重要系统 Atlas [KE+62, L78]。我们不再把进程（process）的地址空间（address space）划分成代码、堆（heap）、栈（stack）等若干个大小可变的逻辑段（logical segment），而是把它划分成大小固定的单元，每个单元叫作一**页**（page）。相应地，我们把物理内存（physical memory）看作由大小固定的槽组成的数组，这些槽叫作**页帧**（page frame）；每个页帧可以容纳虚拟内存的一个页。我们面临的挑战是：

关键问题：如何用页来虚拟化内存
如何用页来虚拟化（virtualize）内存，从而避免分段的那些问题？基本技术有哪些？如何让这些技术运行良好，同时把空间和时间开销降到最低？

18.1 一个简单的例子和概览

为了把这种方法讲得更清楚，我们用一个简单的例子来说明。图 18.1（第 2 页）展示了一个很小的地址空间，总共只有 64 字节，分成 0、1、2、3 四个虚拟页（virtual page），每页 16 字节。当然，真实的地址空间要大得多，通常是 32 位，也就是 4 GB 的地址空间，甚至是 64 位[1]；本书常用很小的例子，好让内容更容易理解。

如图 18.2 所示，物理内存同样由若干个大小固定的槽组成，这里是 8 个页帧，整个物理内存只有 128 字节，同样小得离谱。从图中可以看到，虚拟地址空间（virtual address space）的各个页被放在了物理内存中的不同位置；图中还显示，操作系统自己也占用了一部分物理内存。

[1] 64 位的地址空间大得惊人，很难想象。打个比方也许有帮助：如果把 32 位地址空间看作一个网球场那么大，64 位地址空间就大约有整个欧洲那么大（!）。

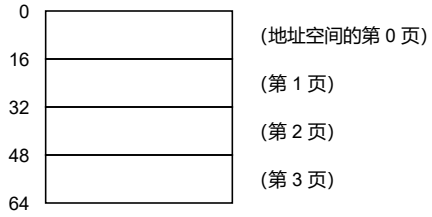


图 18.1：**一个简单的 64 字节地址空间**



图 18.2：**64 字节的地址空间放在 128 字节的物理内存中**

我们将会看到，与之前的方法相比，分页有许多优点。最重要的改进可能是*灵活性*：有了完善的分页机制，无论进程怎样使用地址空间，系统都能有效地支持地址空间这一抽象（abstraction）；例如，我们不必对堆和栈的增长方向以及它们的用法做任何假设。

另一个优点是分页让空闲空间管理（free-space management）变得很简单。例如，当操作系统想把这个 64 字节的小地址空间放进只有 8 个页帧的物理内存时，只需找到四个空闲页（free page）即可；操作系统也许为此维护了一个记录所有空闲页的**空闲链表**（free list），从中取出前四个空闲页就行了。在这个例子中，操作系统把地址空间的虚拟页 0 放在了物理帧（physical frame）3，虚拟页 1 放在物理帧 7，页 2 放在帧（frame）5，页 3 放在帧 2。页帧 1、4、6 目前是空闲的。

为了记录地址空间的每个虚拟页放在物理内存的什么位置，操作系统通常会为每个进程维护一个数据结构（data structure），叫作**页表**（page table）。页表的主要作用是保存地址空间中每个虚拟页的**地址转换**（address translation），让我们知道每个页位于物理内存的哪个位置。对于我们这个简单的例子（第 2 页的图 18.2），页表有以下四项：虚拟页 0 → 物理帧 3、VP 1 → PF 7、VP 2 → PF 5 和 VP 3 → PF 2。

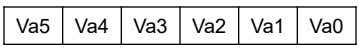
需要记住的是，页表是每个进程各有一份的数据结构。我们讨论的大多数页表结构都是每个进程一份；后面会提到的一个例外是**反置页表**（inverted page table）。如果上面的例子中还有另一个进程要运行，操作系统就得为它管理另一张页表，因为它的虚拟页显然会映射到不同的物理页（physical page），有共享的情况除外。

现在，我们已经可以动手做一个地址转换的例子了。假设那个只有 64 字节地址空间的小进程正在执行一次内存访问：

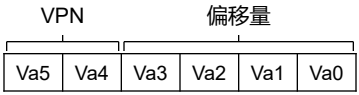
```
movl <virtual address>, %eax
```

具体来说，我们只关注从地址 `<virtual address>` 把数据显式加载到寄存器 `eax` 的过程（因此忽略在此之前必然发生过的取指令）。

为了**转换**（translate）进程产生的这个虚拟地址（virtual address），我们首先要把它拆成两部分：**虚拟页号**（virtual page number，VPN）和页内的**偏移量**（offset）。在这个例子中，进程的虚拟地址空间是 64 字节，所以虚拟地址一共需要 6 位（$2^6 = 64$）。因此，虚拟地址可以看成下面这样：



图中 Va5 是虚拟地址的最高位，Va0 是最低位。我们知道页大小（page size）是 16 字节，因此可以把虚拟地址进一步划分成这样：



地址空间是 64 字节，页大小是 16 字节，所以要能从 4 个页中选出一个，地址的最高 2 位正好用来做这件事。因此，虚拟页号（VPN）是 2 位。剩下的位（这里是 4 位）告诉我们要访问页中的哪个字节，我们把它叫作偏移量。

进程产生一个虚拟地址时，操作系统和硬件必须相互配合，把它转换成有意义的物理地址（physical address）。例如，假设上面那条加载指令访问的是虚拟地址 21：

```
movl 21, %eax
```

把“21”写成二进制，得到“010101”。由此可以看出这个虚拟地址是怎样拆分成虚拟页号（VPN）和偏移量的：



所以，虚拟地址“21”位于虚拟页“01”（即 1）的第 5 个（第“0101”个）字节。有了虚拟页号，我们就可以用它来索引页表，找出虚拟页 1 位于哪个物理帧。在上面的页表中，**物理帧号**（physical frame number，**PFN**；有时也叫**物理页号**，physical page number，**PPN**）是 7（二进制 111）。于是，只需把 VPN 换成 PFN，就能完成这个虚拟地址的转换，然后向物理内存发出加载请求（图 18.3）。

注意，偏移量保持不变（也就是说，它不用转换），因为偏移量只是告诉我们要访问页内的哪个字节。最终的物理地址是 1110101（十进制 117），这正是我们想要加载数据的位置（第 2 页的图 18.2）。

有了这个基本概念，我们就可以提出（但愿也能回答）关于分页你可能会有的几个基本问题了。例如，这些页表存放在哪里？页表里一般有什么，又有多大？分页会让系统变得（太）慢吗？对于这些以及其他一些耐人寻味的问题，下文至少会给出部分答案。接着往下读吧！

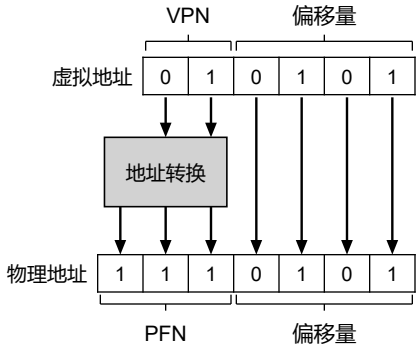


图 18.3：**地址转换过程**

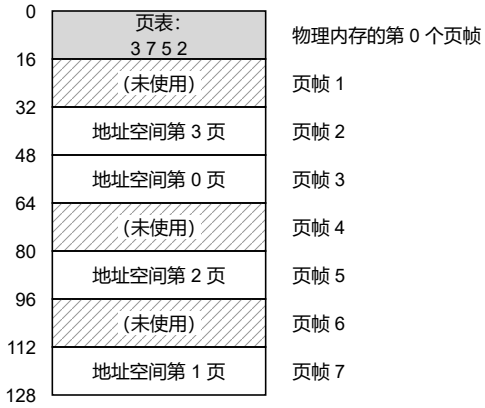


图 18.4：**示例：位于内核物理内存中的页表**

18.2 页表存放在哪里?

页表可能会变得非常大，比我们之前讨论过的小小的段表（segment table）或基址/界限寄存器对（base/bounds pair）大得多。例如，设想一个典型的 32 位地址空间，页大小为 4 KB。这样的虚拟地址会拆分成 20 位的 VPN 和 12 位的偏移量。回想一下，1 KB 的页需要 10 位偏移量，再加两位就是 4 KB。

补充：数据结构——页表

现代操作系统的内存管理子系统中，最重要的数据结构之一就是**页表**（page table）。一般来说，页表保存的是**虚拟地址到物理地址的转换**（virtual-to-physical address translation），让系统知道地址空间的每个页实际位于物理内存的什么地方。由于每个地址空间都需要这样的转换，系统中一般每个进程都有一张页表。页表的具体结构要么由硬件决定（较老的系统），要么可以由操作系统更灵活地管理（现代系统）。

 © 2008–25, ARPACI-DUSSEAU

20 位的 VPN 意味着操作系统要为每个进程管理 2^{20} 个（大约一百万个）地址转换；假设每个**页表项**（page table entry，PTE）需要 4 个字节，用来保存物理转换以及其他有用的信息，那么每张页表就需要整整 4 MB 内存！这已经相当大了。再想象一下有 100 个进程在运行：这意味着光是这些地址转换，操作系统就需要 400 MB 内存！即使在机器动辄拥有好几 GB 内存的今天，拿出其中一大块只用来存放转换信息，似乎也有点疯狂，不是吗？至于 64 位地址空间的页表会有多大，我们连想都不敢想；那实在太可怕了，说不定会把你彻底吓跑。

正因为页表这么大，我们不会在 MMU 中设置专门的片上硬件来存放当前运行进程的页表，而是把每个进程的页表存放在*内存*中的某个地方。我们暂时假设页表位于操作系统管理的物理内存中；后面会看到，操作系统自己的内存有很大一部分也可以被虚拟化，因此页表可以存放在操作系统的虚拟内存中（甚至可以被交换到磁盘上），不过现在讲这些只会让人更糊涂，所以先不管它。图 18.4（第 4 页）展示了操作系统内存中的一张页表；看到里面那一小组转换了吗？

18.3 页表里究竟有什么？

我们来谈谈页表的组织方式。页表就是一种数据结构，用来把虚拟地址映射到物理地址，确切地说，是把虚拟页号映射到物理帧号（physical frame number）。所以，用什么数据结构都可以。最简单的形式叫作**线性页表**（linear page table），它就是一个数组。操作系统用虚拟页号（VPN）来*索引*这个数组，查找该索引处的页表项（PTE），从而找到所需的物理帧号（PFN）。目前我们先假设采用这种简单的线性结构；在后面的章节中，我们会用更高级的数据结构来解决分页的一些问题。

再来看每个 PTE 的内容。PTE 中有许多不同的位，都值得了解一下。常见的一种是**有效位**（valid bit），表示某个转换是否有效。例如，程序开始运行时，代码和堆位于地址空间的一端，栈位于另一端。中间所有未使用的空间都会被标记为**无效**（invalid）；如果进程试图访问这样的内存，就会陷入操作系统，而操作系统很可能会终止这个进程。因此，有效位对于支持稀疏地址空间至关重要：只需把地址空间中所有未使用的页标记为无效，就不必为这些页分配物理帧，从而节省大量内存。

PTE 中还可能有**保护位**（protection bits），表示这个页能否读取、写入或执行。同样，以这些位不允许的方式访问页，就会陷入操作系统。

还有几个位也很重要，不过暂时不细谈。**存在位**（present bit）表示这个页是在物理内存中，还是已被**换出**（swapped out）到磁盘上。等到我们研究如何把地址空间的一部分**交换**（swap）到磁盘、以支持比物理内存更大的地址空间时，会进一步了解这套机制；交换

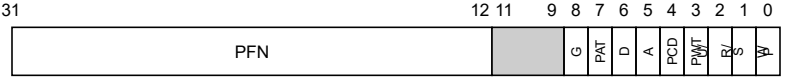


图 18.5：**一个 x86 页表项（PTE）**

补充：为什么没有有效位?

你可能会注意到，在 Intel 的例子中，没有单独的有效位和存在位（present bit），只有一个存在位（P）。如果这一位为 1（P=1），就表示这个页既存在又有效。如果为 0（P=0），则表示这个页可能不在内存中（但它是有效的），也可能是无效的。访问 P=0 的页会陷入操作系统；操作系统必须借助它自己维护的其他结构，判断这个页是有效的（那也许就应该把它换回内存），还是无效的（那就说明程序在试图非法访问内存）。这种精打细算在硬件中很常见：硬件往往只提供最基本的一组功能，操作系统在此基础上构建完整的服务。

让操作系统可以把很少使用的页移到磁盘上，从而腾出物理内存。**脏位**（dirty bit）也很常见，表示这个页自从调入内存以来是否被修改过。

访问位（**reference bit**，又称 **accessed bit**）有时用来跟踪一个页是否被访问过，可以帮助判断哪些页比较常用、应该留在内存中；这些信息在**页面置换**（page replacement）时至关重要，后续章节会详细研究这个话题。

图 18.5 展示了 x86 架构的一个页表项示例 [I09]。它包含一个存在位（P）；一个读/写位（R/W），决定是否允许写这个页；一个用户/超级用户位（user/supervisor bit，U/S），决定用户模式（user mode）的进程能否访问这个页；几个位（PWT、PCD、PAT 和 G），决定硬件如何缓存（cache）这些页；一个访问位（A）和一个脏位（D）；最后是页帧号（PFN）本身。

想了解 x86 分页支持的更多细节，请阅读 Intel 架构手册 [I09]。不过要先提醒你：这类手册虽然信息量很大，要在操作系统里写代码使用这类页表的人当然非读不可，但刚开始读可能比较吃力。你需要一点耐心，外加十足的求知欲。

18.4 分页：不仅太大，而且太慢

我们已经知道，放在内存中的页表可能太大。其实，它们还会拖慢速度。以这条简单的指令为例：

```
movl 21, %eax
```

同样，我们只看对地址 21 的显式引用，不考虑取指令。在这个例子中，我们假设由硬件替我们完成转换。为了取出所需的数据，系统必须先把虚拟地址 21 **转换**成正确的物理地址 117。因此，在从地址 117 取数据之前，系统必须先从进程的页表中取出相应的页表项，完成转换，然后才能从物理内存加载数据。

为此，硬件必须知道当前运行进程的页表在哪里。我们暂时假设有一个**页表基址寄存器**（page-table base register），里面存放着页表起始位置的物理地址。为了找到所需 PTE 的位置，硬件会执行以下计算：

```
VPN      = (VirtualAddress & VPN_MASK) >> SHIFT
PTEAddr = PageTableBaseRegister + (VPN * sizeof(PTE))
```

 © 2008–25, ARPACI-DUSSEAU

```
1  // 从虚拟地址中提取 VPN
2  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
3
4  // 计算页表项（PTE）的地址
5  PTEAddr = PTBR + (VPN * sizeof(PTE))
6
7  // 取出 PTE
8  PTE = AccessMemory(PTEAddr)
9
10 // 检查进程能否访问这个页
11 if (PTE.Valid == False)
12     RaiseException(SEGMENTATION_FAULT)
13 else if (CanAccess(PTE.ProtectBits) == False)
14     RaiseException(PROTECTION_FAULT)
15 else
16     // 访问合法：计算物理地址并取数据
17     offset   = VirtualAddress & OFFSET_MASK
18     PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
19     Register = AccessMemory(PhysAddr)
```

图 18.6：**使用分页访问内存**

在我们的例子中，VPN_MASK 会被设为 0x30（十六进制 30，即二进制 110000），用来从完整的虚拟地址中挑出 VPN 位；SHIFT 设为 4，即偏移量的位数，这样就能把 VPN 位移到低位，得到正确的整数虚拟页号。例如，对于虚拟地址 21（010101），掩码运算把它变成 010000，移位再把它变成 01，也就是虚拟页 1，和预期一致。然后，我们用这个值作为索引，在页表基址寄存器（page-table base register）指向的 PTE 数组中查找。

知道了这个物理地址，硬件就可以从内存中取出 PTE，从中提取 PFN，再把它和虚拟地址中的偏移量拼接起来，得到所需的物理地址。具体来说，可以理解为把 PFN 左移 SHIFT 位，再和偏移量按位或，就得到最终地址，如下所示：

```
offset   = VirtualAddress & OFFSET_MASK
PhysAddr = (PFN << SHIFT) | offset
```

最后，硬件就可以从内存中取出所需的数据，放进寄存器 eax。程序终于成功地从内存中加载了一个值！

总结一下，图 18.6（第 7 页）给出了每次内存引用时的初步协议（protocol）。每次内存引用（无论是取指令，还是显式的加载或存储），分页都要额外再做一次内存引用，先从页表中取出转换。这可是很大的工作量！额外的内存引用代价高昂，在这种情况下，很可能会让进程慢一倍甚至更多。

现在，你应该能看出我们必须解决的两个实实在在的问题了。如果不精心设计硬件和软件，页表既会让系统运行得太慢，又会占用太多内存。分页看起来是满足内存虚拟化需求的绝佳方案，但必须先克服这两个关键问题。

18.5 一次内存跟踪

在结束本章之前，我们来跟踪一个简单的内存访问例子，展示使用分页时产生的全部内存访问。我们关心的代码片段（用 C 语言写的，在一个名为 `array.c` 的文件中）如下：

```
int array[1000];
...
for (i = 0; i < 1000; i++)
    array[i] = 0;
```

用下面的命令编译并运行 `array.c`：

```
prompt> gcc -o array array.c -Wall -O
prompt> ./array
```

当然，要真正弄清这段代码（它只是初始化一个数组）会产生哪些内存访问，我们还得知道（或者假设）一些其他信息。首先，我们得**反汇编**（disassemble）生成的二进制文件（在 Linux 上用 `objdump`，在 Mac 上用 `otool`），看看循环初始化数组用的是哪些汇编指令。下面是得到的汇编代码：

```
1024 movl $0x0,(%edi,%eax,4)
1028 incl %eax
1032 cmpl $0x03e8,%eax
1036 jne  1024
```

如果你懂一点 **x86**，这段代码其实很容易理解[2]。第一条指令把值 0（写作 `$0x0`）写到数组所在位置的虚拟内存地址；这个地址的计算方法是：`%edi` 的内容加上 `%eax` 乘以 4。因此，`%edi` 保存的是数组的基址（base），而 `%eax` 保存的是数组下标（`i`）；之所以乘以 4，是因为这是一个整数数组，每个元素占 4 个字节。

第二条指令把 `%eax` 中保存的数组下标加 1，第三条指令把这个寄存器的内容与十六进制值 `0x03e8`（即十进制 1000）比较。如果比较结果表明两个值还不相等（这正是 `jne` 指令要检测的），第四条指令就跳回循环的开头。

为了弄清这段指令序列会产生哪些内存访问（包括虚拟层面和物理层面），我们必须对代码片段和数组在虚拟内存中的位置，以及页表的内容和位置做一些假设。

在这个例子中，我们假设虚拟地址空间大小为 64 KB（小得不现实），页大小为 1 KB。

现在只需要知道页表的内容和它在物理内存中的位置。假设页表是线性的（基于数组），位于物理地址 1 KB（1024）处。

至于页表的内容，这里只需关心少数几个虚拟页的映射。首先是代码所在的虚拟页。由于页大小为 1 KB，虚拟地址 1024 位于虚拟地址空间的第二个页（VPN=1，因为 VPN=0 是第一个页）。假设这个虚拟页映射到物理帧 4（VPN 1 → PFN 4）。

其次是数组本身。它的大小是 4000 字节（1000 个整数），我们假设它位于虚拟地址 40000 到 44000（不含最后一个字节）。这个十进制范围对应的虚拟页是 VPN=39 … VPN=42。因此，我们需要为这些页建立映射。在这个例子中，假设虚拟地址到物理地址的

[2]这里我们稍微作了点弊：为简单起见，假设每条指令都是 4 字节；实际上，x86 指令的长度是可变的。

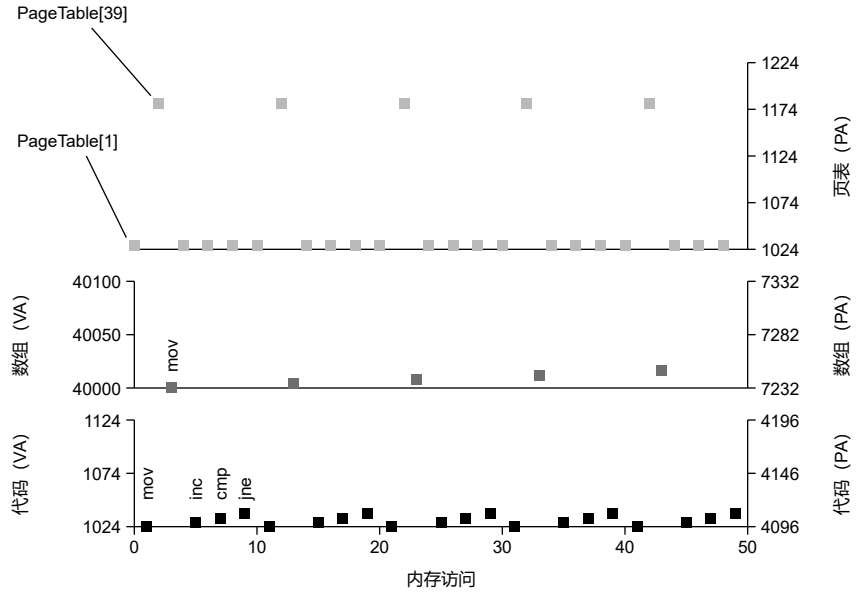


图 18.7：**一次虚拟（及物理）内存跟踪**

映射如下：（VPN 39 → PFN 7）、（VPN 40 → PFN 8）、（VPN 41 → PFN 9）、（VPN 42 → PFN 10）。

现在可以跟踪程序的内存引用了。程序运行时，每次取指令都会产生两次内存引用：一次访问页表，找到指令所在的物理帧；另一次访问指令本身，把它取到 CPU 中处理。此外，`mov` 指令还有一次显式的内存引用；这又会先增加一次页表访问，把数组的虚拟地址转换成正确的物理地址，然后才是对数组本身的访问。

图 18.7（第 9 页）描绘了前五次循环迭代的整个过程。最下面的图用黑色在 y 轴上标出指令的内存引用，左边是虚拟地址，右边是实际的物理地址；中间的图用深灰色标出数组访问，同样是左边虚拟地址、右边物理地址；最上面的图用浅灰色标出页表的内存访问，只有物理地址，因为在这个例子中页表位于物理内存中。x 轴是整个跟踪过程中循环前五次迭代的内存访问；每次循环有 10 次内存访问，包括四次取指令、一次显式的内存更新，以及为这五次访问做地址转换的五次页表访问。

看看你能否理解这张图中呈现的规律。特别是，循环在前五次迭代之后继续运行时，会有什么变化？会访问哪些新的内存位置？你能想出来吗？

这只是最简单的例子（仅几行 C 代码），但你或许已能感到，理解真实应用的内存行为有多复杂。别担心：肯定会更糟，因为即将介绍的机制只会让本已复杂的系统更复杂。抱歉[3]！

[3]我们其实并不觉得抱歉。不过，我们为不觉得抱歉而感到抱歉，如果这说得通的话。

18.6 小结

本章介绍了**分页**（paging）的概念，它是应对内存虚拟化挑战的一种方案。与分段等之前的方法相比，分页有许多优点。首先，它不会产生外部碎片（external fragmentation），因为分页在设计上就把内存划成了大小固定的单元。其次，它很灵活，允许稀疏地使用虚拟地址空间。

然而，分页实现得不好，机器就会因为访问页表要多出很多内存访问而变慢，内存也会因为塞满页表而不是有用的应用程序数据而被浪费。因此，我们得多想想，设计出既能用又好用的分页系统。好在下两章会讲怎么做。

 © 2008–25, ARPACI-DUSSEAU

参考文献

[KE+62] “One-level Storage System” by T. Kilburn, D.B.G. Edwards, M.J. Lanigan, F.H. Sumner. IRE Trans. EC-11, 2, 1962. Reprinted in Bell and Newell, “Computer Structures: Readings and Examples”. McGraw-Hill, New York, 1971. *The Atlas pioneered the idea of dividing memory into fixed-sized pages and in many senses was an early form of the memory-management ideas we see in modern computer systems.*

[I09] “Intel 64 and IA-32 Architectures Software Developer’s Manuals” Intel, 2009. Available: http://www.intel.com/products/processor/manuals. *In particular, pay attention to “Volume 3A: System Programming Guide Part 1” and “Volume 3B: System Programming Guide Part 2”.*

[L78] “The Manchester Mark I and Atlas: A Historical Perspective” by S. H. Lavington. Communications of the ACM, Volume 21:1, January 1978. *This paper is a great retrospective of some of the history of the development of some important computer systems. As we sometimes forget in the US, many of these new ideas came from overseas.*

 © 2008–25, ARPACI-DUSSEAU

作业（模拟）

在这个作业中，你将使用一个名为`paging-linear-translate.py`的简单程序，检验自己是否理解了使用线性页表时，虚拟地址到物理地址的简单转换是怎样进行的。详见 README。

问题

1. 在做任何转换之前，我们先用模拟器研究一下，在不同参数下线性页表的大小如何变化。改变参数，算出相应的线性页表大小。下面是一些建议的输入；使用 `-v` 选项，可以看到有多少页表项被填充了。首先，为了理解线性页表的大小如何随地址空间的增长而变化，用下面这些选项运行：
   - `-P 1k -a 1m -p 512m -v -n 0`
   - `-P 1k -a 2m -p 512m -v -n 0`
   - `-P 1k -a 4m -p 512m -v -n 0`
2. 接下来，看看线性页表的大小如何随页大小的增长而变化。运行这些命令之前，先想想预期的趋势。随着地址空间的增长，页表大小应该如何变化？随着页大小的增长呢？为什么一般不使用很大的页？
   - `-P 1k -a 1m -p 512m -v -n 0`
   - `-P 2k -a 1m -p 512m -v -n 0`
   - `-P 4k -a 1m -p 512m -v -n 0`
3. 现在来做一些转换。从一些小例子开始，并用 `-u` 选项改变分配给地址空间的页数。增大每个地址空间中已分配页所占的比例，会发生什么？
   - `-P 1k -a 16k -p 32k -v -u 0`
   - `-P 1k -a 16k -p 32k -v -u 25`
   - `-P 1k -a 16k -p 32k -v -u 50`
   - `-P 1k -a 16k -p 32k -v -u 75`
   - `-P 1k -a 16k -p 32k -v -u 100`
4. 现在换换花样，试试一些不同的随机种子，以及一些不同的（有时相当疯狂的）地址空间参数。这些参数组合中，哪些是不现实的？为什么？
   - `-P 8 -a 32 -p 1024 -v -s 1`
   - `-P 8k -a 32k -p 1m -v -s 2`
   - `-P 1m -a 256m -p 512m -v -s 3`
5. 用这个程序试试其他一些问题。你能找出程序在什么情况下就不能正常工作了吗？例如，如果地址空间比物理内存大，会发生什么？

 © 2008–25, ARPACI-DUSSEAU